Report - BE-SHS

Project BE — Phase 1

Maximilien Larroche 22301310

---

1. Introduction

My project is to compare several data's structure of int in rust with the same trait. The first part is alone. I have to use the assemblist function : union, intersection, difference, symetric difference, also serialisation and deseralisation and fragmentation. The second half we have to compare the trait and result by group and decide one trait for everyone to continue.

2. My research

2.1. First Implementation

First of all, I don't know to code in rust, it was the most of my time. To learn I choose to watch video on Youtube and do the rust book. After that I start to build a trait with the basic function like new, add, remove, delete, there_is and also the protype of fragmenter. I start to implement the binarytree structure because it was the structure I know the most, it was difficult cause to the syntax of rust with the ownership system, mutable type and the gestion of references. In the process I discover that rust have a function drop that free the memory so the function delete wasn't necessary. I also add the prototype of map and an iterator.

```
pub trait StructureDonnee {
    fn new() -> Self;
    fn add(&mut self, value: i32);
    fn remove(&mut self, value: i32);
    fn there_is(&self, value: i32) -> bool;
    fn map(&self, f: impl Fn(i32) -> i32 + Copy) -> Self;
    fn iter(&self) -> Box<dyn Iterator<Item = &i32> + '_>;
    fn fragmenter(self, taille_max: usize) -> Vec<Self>
    where
        Self: Sized + Clone;
}
```

After that I decide to implement the ensemblist function in the trait that I don't need to do it in every structure. I was able to do it with add, remove, there_is.

2.2. Test

I have to test my code for the performance, in Rust there is a bibliotheque call criterion that measure the time of a function. I decide to implement that in a generic way to not duplicate code for every structure that I test.